

Szoftvertchnológia és -technikák

8. Előadás – Benedek Zoltán
Tervezési minták 3



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési minták

- > Command
- > Command Processor
- > Memento
- > Adapter
- > Composite

Adapter (Illesztő)

(Wrapper)

Adapter

- Cél

➤ Egy osztály interfészét olyan interfésszé konvertálja, amilyent a kliens vár. Lehetővé teszi olyan osztályok együttműködését, melyek egyébként az inkompatibilis interfészeik miatt nem tudnának együttműködni.

- Alternatív név: Wrapper

- Elve



?



Nem megfelelő interfész



Megoldás: adapter

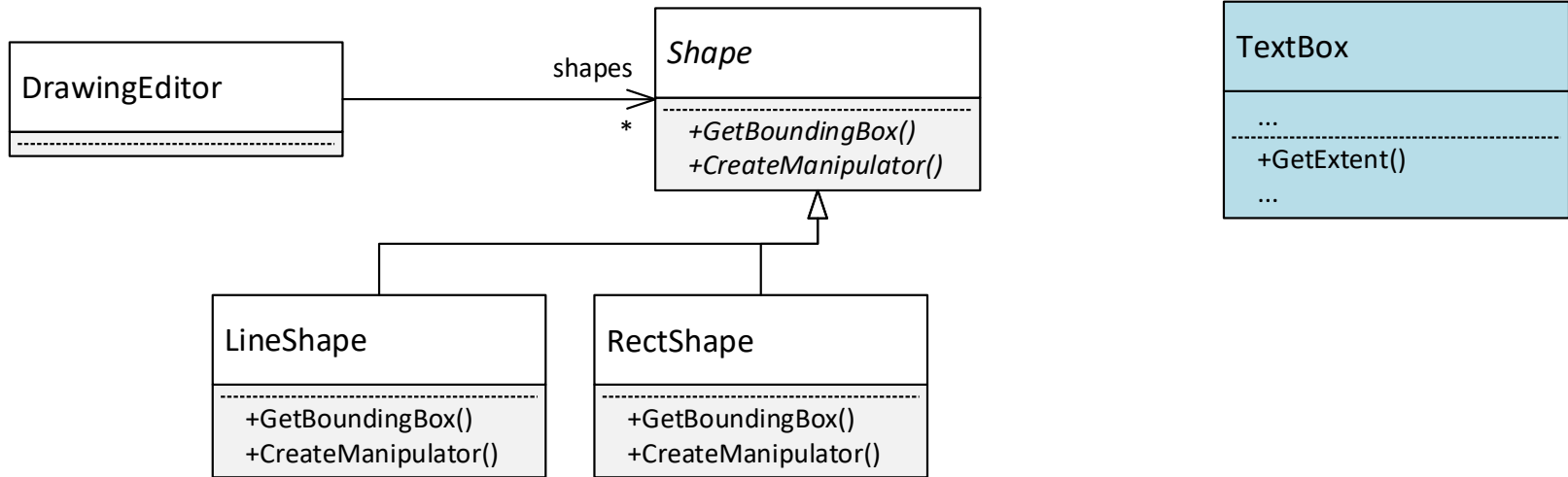
Adapter - példa

- A feladatunk egy vektorgrafikus alkalmazás elkészítése
 - > A felhasználó különböző grafikus alakzatokat tud elhelyezni a felületen. Támogatni kell a vonal, téglalap, stb. alakzatot, valamint a **szerkeszthető szövegdobozt** is (amibe lehet gépelni futás közben)
 - > Bevezetünk egy **Shape** **őszosztályt**, hogy az **alakzatokat egységesen szeretnék kezelni** (esetünkben **egy heterogén kollekcióban tudjuk tárolni**). A grafikus alakzatokat ebből a Shape osztályból származtatjuk. Pl. LineShape, RectShape, EditableTextShape.
 - > **Probléma:** az EditableTextShape (egy szerkeszthető szövegdoboz teljes logikájának) megvalósítása nagyon-nagyon nehéz, beláthatatlan mennyiségű munka.

Adapter – példa folytatás

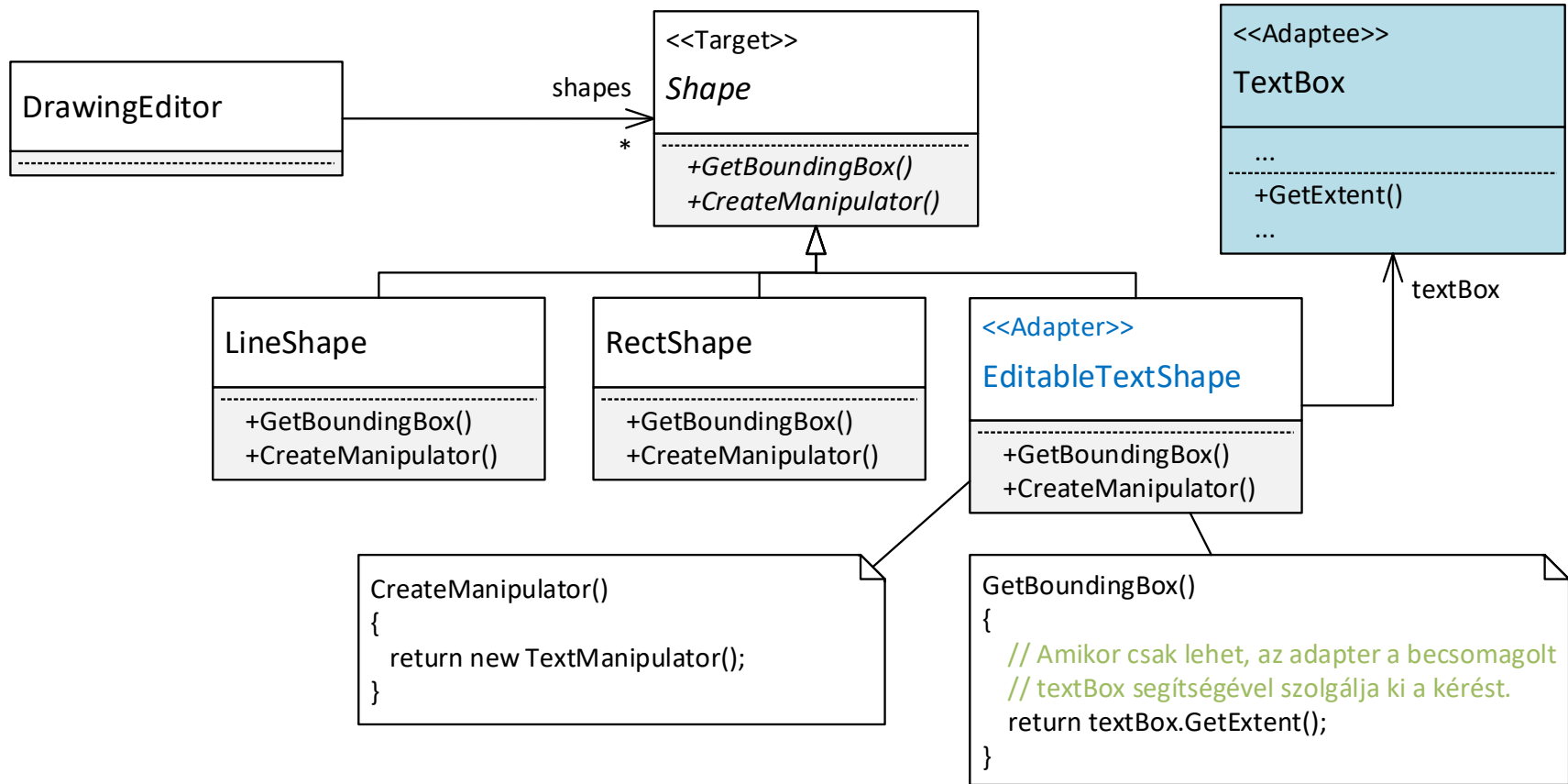
- > T.f.h. az alkalmazást egy adott keretrendszerre (pl. UWP) építve írjuk, amiben **már van egy beépített szerkeszthető szövegdoboz** (TextBox osztály), ami tudja mindazt, amit az EditableTextShape-től elvárunk. Jó lenne ezt felhasználni.
- > Probléma: a **beépített TextBox osztályt nem tudjuk közvetlenül felhasználni, mert nem megfelelő az interfésze**, ugyanis nem a Shape osztályból származik (emiattnem tudjuk a többi Shape-el együtt egységesen kezelni). Mivel egy „beépített” osztály, a forráskódját nem is tudjuk módosítani (hogya Shape osztályból származzon)
- > Probléma: nem tudjuk újrafelhasználni a meglévő osztályt!
- > Megoldás: Adapter minta használata (Object Adapter vagy Class Adapter)

Kiindulás

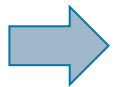


- A **DrawingEditor** egy **Shape** listát tárol (heterogén kollekció), a **TextBox** nem tehető bele
- A **GetBoundingBox** művelet egy befoglaló téglalapot ad vissza
- A **CreateManipulator** egy „manipulátor” objektumot, amivel az adott típusú alakzat szerkeszthető (pl. átméreteztető, stb., nincs jelentősége a példában)

Példa megoldás Object Adapterrel



Magyarázat



Példa megoldás Object Adapterrel

- Az eredeti tervünknek megfelelően leszarmaztatunk az `Shape` osztályból (vagy, ha a `Shape` interfész, implementáljuk azt) → Ez lesz az `EditableTextShape` osztály. Így az `EditableTextShape`-nek jó lesz az interfésze, a `DrawingEditor` tudja ezt is kezelni.
- Az `EditableTextShape` nem maga valósítja meg a műveletek többségét. Helyette:
 - > Példányosít és becsomagol egy `TextBox` osztályt
 - > A műveletek többségénél továbbhív a `TextBox` osztályba, delegálja a kéréseket. A példában a `GetBoundingBox` művelet a `TextBox` `GetExtent`-et hívja.
- Így a `EditableTextShape` fel tudja használni a már meglévő komplex logikát (`TextBox` osztály)
- Megjegyzés: a példában a `Shape` egy absztrakt osztály, lehetne interfész is, nincs jelentősége.

Példa megoldás Object Adapterrel

- Kód: lásd DesPattCode/Adapter mappa
- A legfontosabb rész maga az adapter

```
class EditableTextShape : Shape
{
    // A becsomagolt/adaptálandó osztály
    TextBox textBox;

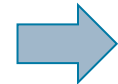
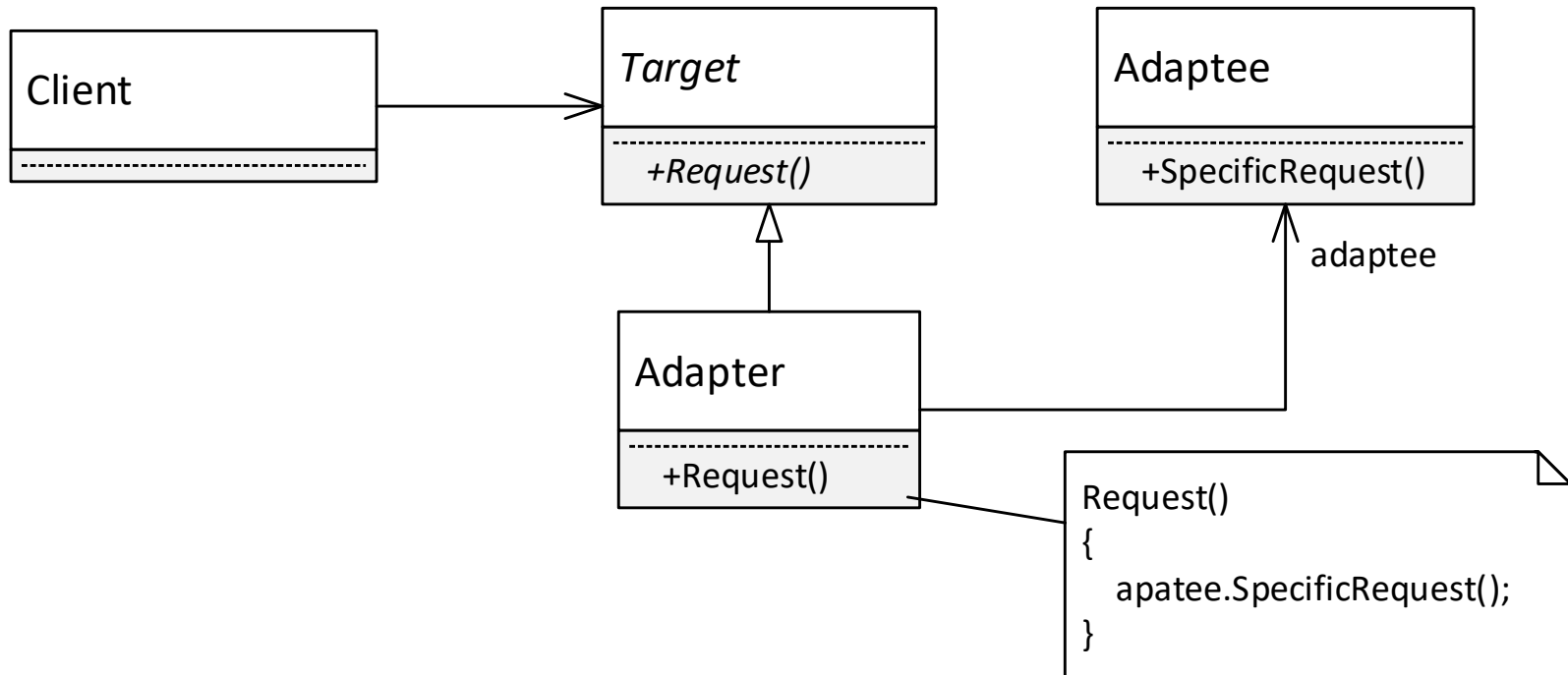
    // ...

    protected override Rect GetBoundingBox()
    {
        // Ez a lelke, ahol csak lehet, továbbítjuk a kérést
        // a becsomagolt/adaptálandó osztálynak, felhasználjuk
        // a kódját
        return textBox.GetExtent();
    }
}
```

Adapter

- Adapter minta általánosságában
 - Két változat
 - > Object Adapter
 - > Class Adapter
- Object Adapter → kompozícióval (has-a)
Class Adapter → örökléssel (is-a)

Object Adapter struktúra

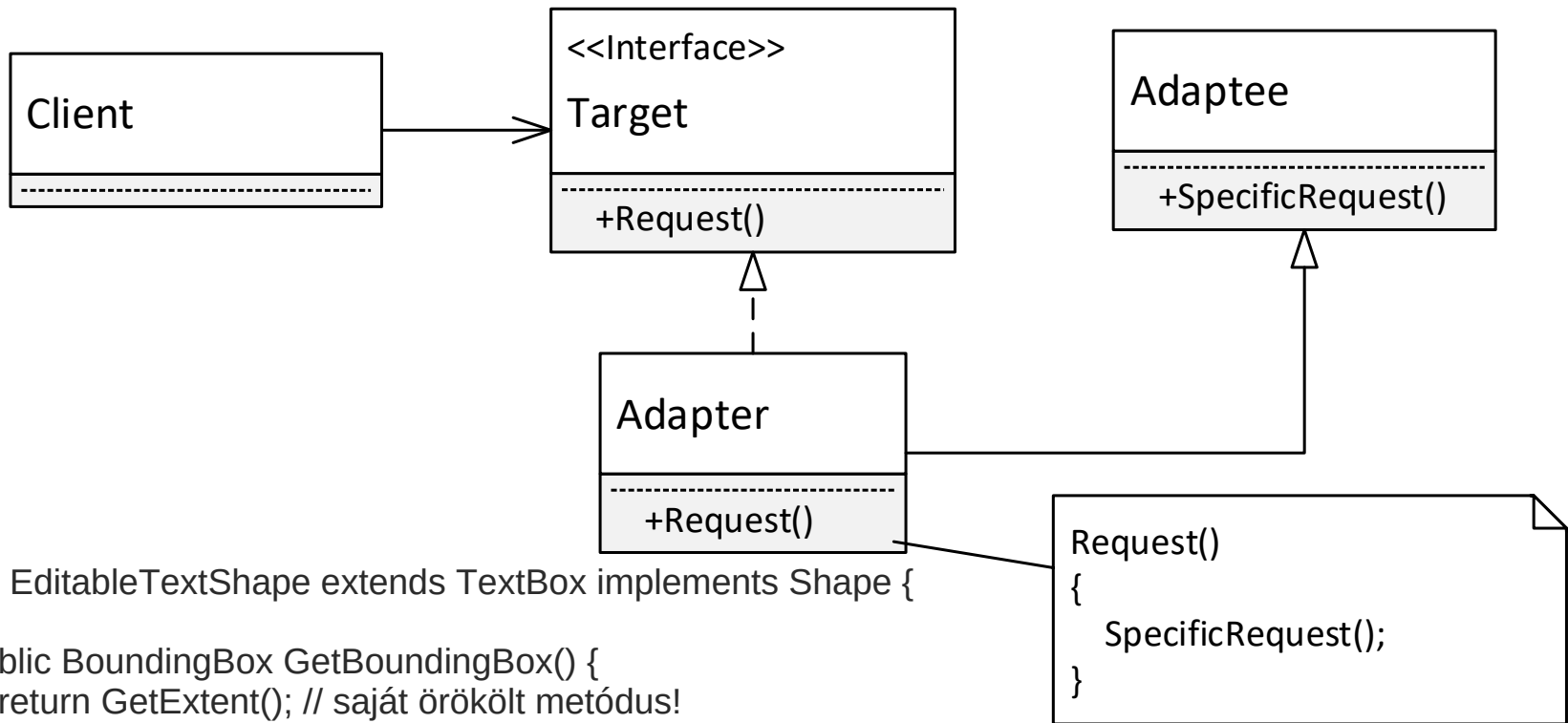


Magyarázat

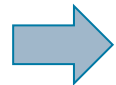
Object Adapter magyarázat

- **Adaptee**: Az adaptálandó osztály, melynek nem megfelelő az interfésze (emiat nem tudjuk az adott környezetben triviális egyszerűséggel felhasználni).
- **Adapter** (*EditableTextShape*) : Illesztő, az Adaptee interfészt a Target interfésszé „konvertálja”.
Tartalmaz egy hivatkozást egy adaptee objektumra (becsomagol egyet). A műveletei, ahol csak tehetik, a becsomagolt adaptee műveleteit hívják.
- **Target** (*Shape*): Interfész, amit a kliens használ. A gyakorlatban lehet interfész vagy absztrakt osztály is.

Class adapter struktúra



Alapja: tartalmazás (becsomagolás)
helyett leszármaztatás.



Magyarázat

Class Adapter magyarázat

- A szerepek alapvetően megegyeznek az Object Adapterével, ami különbség:
 - > Az Adapter nem becsomagolással használja fel az Adaptee kódját, hanem leszármaztatással. Az Adapter műveletei, ahol csak tehetik, az ősz Adaptee műveleteit hívják.
 - > Mivel a legtöbb nyelv (pl. Java, C#) nem támogatja a többszörös öröklést, a Target itt csak interfész lehet, őssztály nem!
 - > Kevésbé rugalmas, mint az Object Adapter, általában az Object Adapter változatot preferáljuk!

Adapter összefoglaló

- Használjuk: ha nem tudunk valamilyen osztályt újrafelhasználni egy környezetben, mert nem jó az interfésze
- Megjegyzések
 - > (Az Object Adapter esetében nem minden esetben szükséges tagváltozóként tárolni az Adaptee-t, ritkább esetben az Adapter művelete maga példányosítja majd el is dobja az Adaptee objektumot. Ekkor nem asszociáció, hanem csak függőség van az Adapter és az Adaptee között.)

Composite (Összetett)

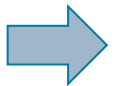
Composite

- Célja

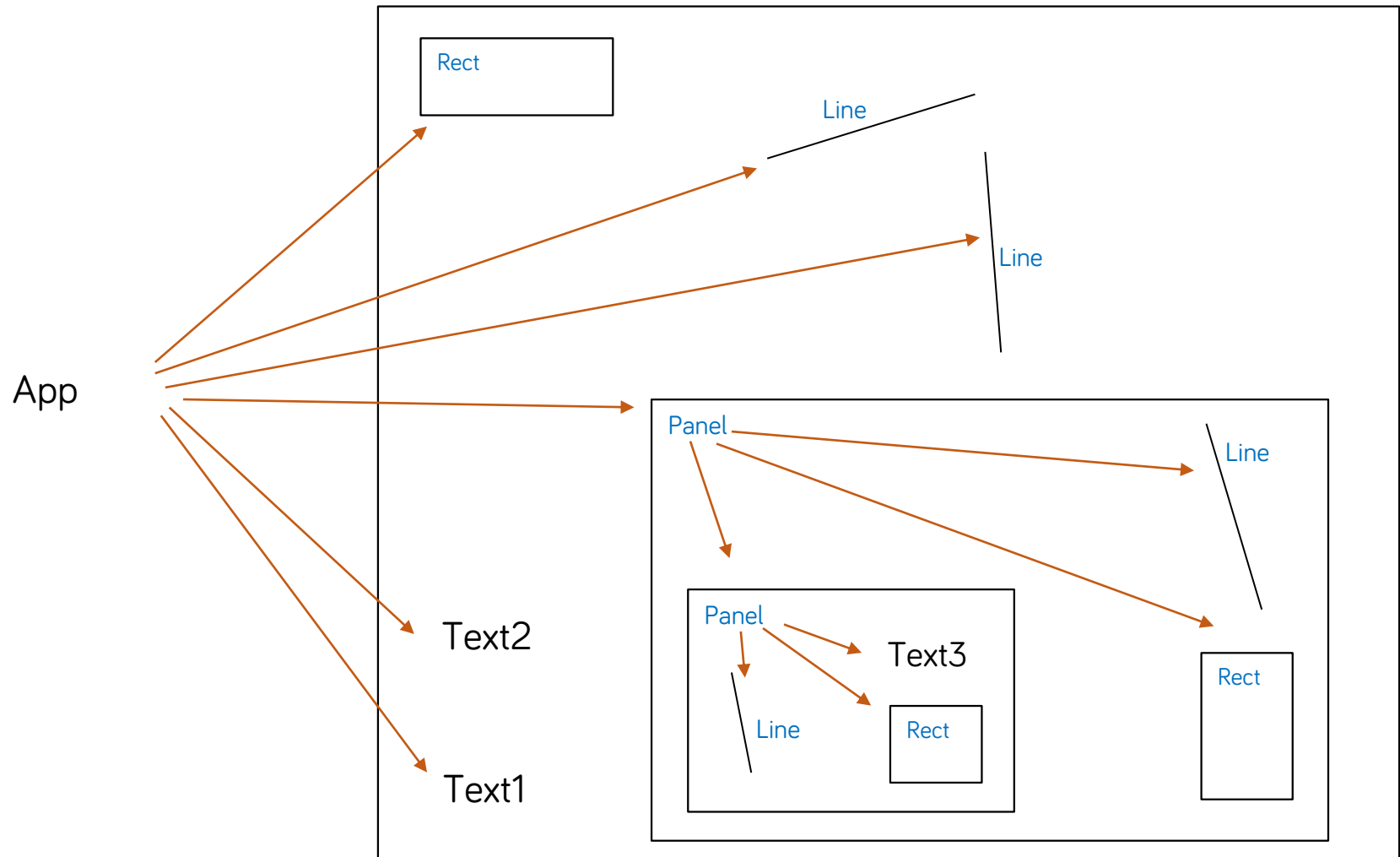
- > Rész-egész viszonyban álló objektumokat fastruktúrába rendezi
- > A kliensek számára lehetővé teszi, hogy az egyszerű és összetett (kompozit) objektumokat egységesen kezelje

- Példa

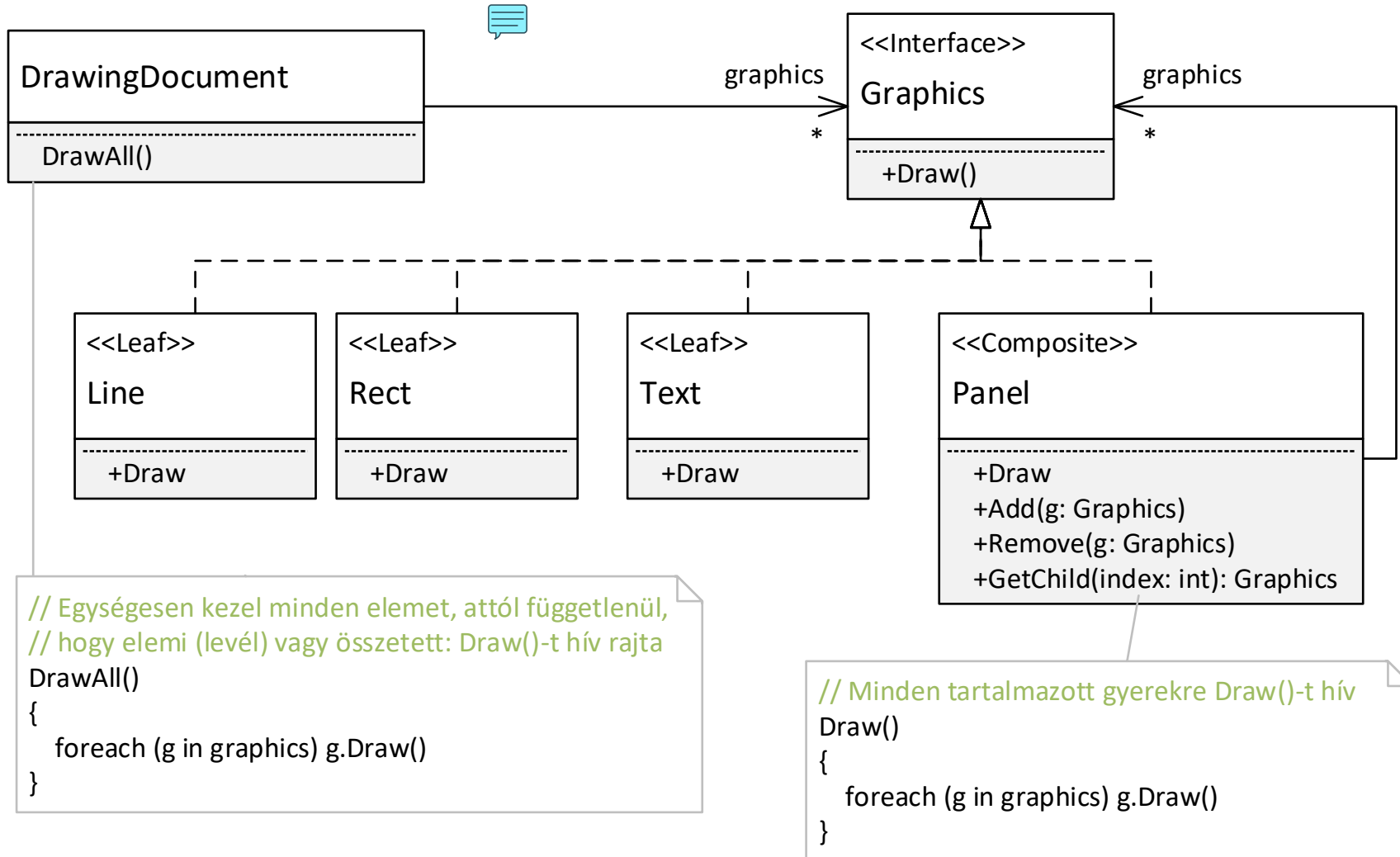
- > Olyan grafikus alkalmazás, amely lehetővé teszi elemi és összetett grafikus objektumokat tartalmazó rajzok létrehozását
- > A Panel egy olyan összetett grafikus elem, mely grafikus alakzatokat tartalmaz (tetszőlegesenek, akár más Panel objektumok is lehetnek rajta!)



Composite példa



Composite példa



Magyarázat



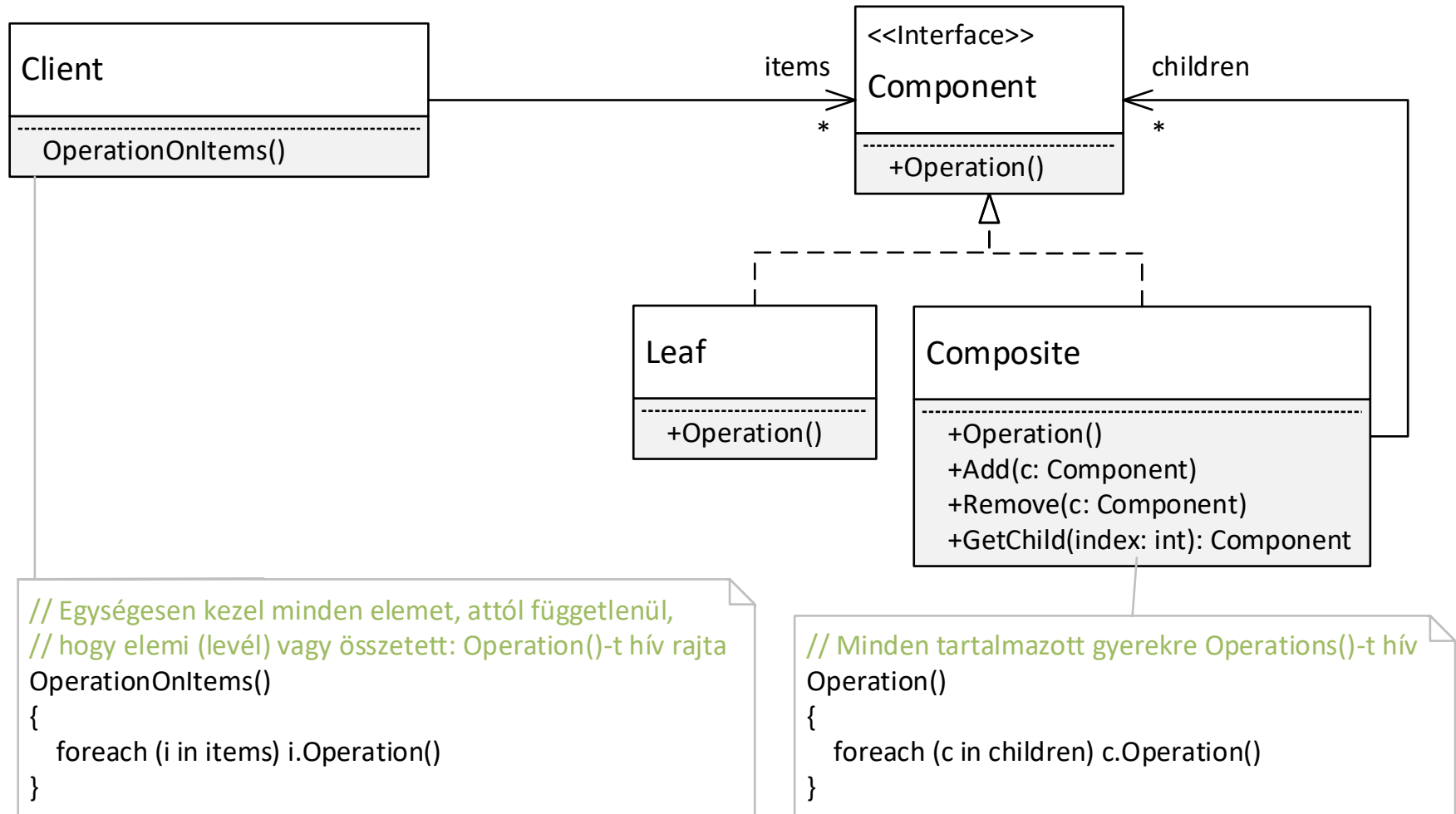
Composite példa magyarázat

- **DrawingDocument:** egy olyan dokumentumot reprezentál, mely különböző grafikus alakzatokat tárol és kezel
- Ezen grafikus alakzatok lehetnek
 - > **Elemiek** (levél, leaf), mint pl. Line, Rect, Text.
 - > **Összetettek**, mint pl. a Panel (mely maga is tartalmazhat elemi és összetett alakzatokat).
- **Graphics:** a grafikus alakzatok közös interfésze/őse, attól függetlenül, hogy elemi vagy összetett az alakzat

Composite példa magyarázat

- A Composite minta egyik alapelve, hogy **egységesen kezeli az elemi (Line, Rect, Text) és az összetett (Panel) objektumokat**. Ez a példában két pontban is megjelenik:
 - > A DrawingDocument **közös gyűjteményben tárolja az elemi és összetett objektumokat** (ez a graphics tag). Megteheti mert az összetett Panel is implementálja a Graphic interfészt.
 - > A DrawAll műveletben a kirajzoló kód **nem különböztetni meg – vagyis egységesen kezeli - az elemi és összetett objektumokat** (nincsenek típus szerint leválogatva): mindre egységesen a Draw() műveletet hívja.
- A összetett Panel Draw művelete az általa tartalmazott alakzatokat rajzolja ki, pont ez volt a célunk!
- Kód: lásd DesPattCode/Composite mappa

Composite általánosságában



Composite

- Használjuk, ha
 - > Objektumok rész-egész viszonyát szeretnénk kezelni (jellemzően fastruktúrában)
 - > A kliensek számára el akarjuk rejtetni, hogy egy objektum elemi objektum vagy kompozit objektum: bizonyos műveletek szempontjából egységesen szeretnénk kezelni őket

Composite megjegyzések

- A gyerekek kezelése nem tud egységes lenni!
 - > Hiszen az csak az összetett (példánkban a Panel) osztály objektumaira értelmezett (Add, Remove, GetChild)
 - > A kliens pl. típusellenőrzéssel tudja megnézni, hogy az adott elem összetett-e (pl. C#-ban az „is”, Java-ban az „instanceof” operátorral), pl.:
 - if (item is Composite) ...
 - > Az Add, Remove, GetChild műveletek hívásához a kliensnek a kompozit osztályra/interfészre kell castolnia a hivatkozását

Tervezési minták összefoglaló

További tervezési minták

- A klasszikus „GoF” minták közül is kimaradt pár:
 - > Prototype, Builder, Bridge, Mediator, Chain of Responsibility, Visitor, Decorator, Iterator, State, stb.
- Számos nem általános tervezési minta létezik, pl.
 - > Elosztott, konkurens rendszerekre jellemző minták
 - Szolgáltatás hozzáférés
 - Konfiguráció
 - Esemény kezelés
 - Szinkronizáció
 - Konkurencia
 - > Valós idejű rendszerekre jellemző minták
 - > Vállalati rendszerekre jellemző minták

Összefoglalás

- Tapasztalati tudást hordoznak
 - > Mi is rá tudunk jönni, de
 - Jó sokáig tart
 - Nem jövünk rá
 - Miért ne tanuljunk mások tapasztalataiból
 - > Értékes tudás!
- Célunk
 - > Ismerjünk meg minél több mintát
 - > Hosszútávon legalább arra emlékezzünk, hogy egy adott problémakörben mely mintákat lehet jól használni